



Guia Estratégico para 20/20 no Exame de PP

Objetivo: Garantir os **6 valores da Parte Teórica** (com respostas diretas e exemplos) e os **14 valores da Parte Prática** (desenvolvimento em Java sem `java.util`).



1. Estratégia para a Parte Teórica (6 Valores)

A parte teórica costuma ter **4 perguntas** de 1,5 valores cada. Os professores de PP corrigem com critérios muito rígidos.



A Formula Mágica da Resposta Perfeita:

Toda a resposta teórica deve seguir rigorosamente a estrutura em **3 partes**:

- ```
[1. Resposta Direta e Concisa (2 a 3 frases definindo a essência)]
[2. Tabela ou Bullet Points Comparativos (Distinção de conceitos)]
[3. Exemplo Mínimo em Código Java (5 a 8 linhas demonstrativas)]
```

### ✗ O que FAIXA PONTOS na Teoria:

1. **Escrever "palha" / texto longo e vago:** O professor quer ir direto ao ponto. Se a pergunta pede a diferença entre `==` e `equals`, não gaste tempo a falar da história do Java.
2. **Esquecer o Exemplo em Código:** Mesmo que a pergunta não diga explicitamente "dê um exemplo", **DAR UM EXEMPLO DE CÓDIGO É OBRIGATÓRIO** para ter a cotação máxima (1,5v).
3. **Usar `java.util` no Exemplo:** Usar `ArrayList` ou `Objects.equals()` resulta em perda imediata de pontos por violação das regras da disciplina.



## 2. Estratégia para a Parte Prática (14 Valores)

A parte prática exige a implementação de classes, interfaces e métodos algorítmicos sobre o **Trabalho Prático**.



### As 6 Regras de Ouro do Código em Exame:

#### 1 Prevenção Absoluta de `NullPointerException`

Em exames written/práticos, se o seu código arriscar um `NullPointerException`, o professor desconta vários décimos.

- **Sempre que receber um array ou objeto por parâmetro:** Valide se é `null`.

- **Sempre que iterar um array:** Verifique se a posição `arr[i] != null` .

```
// ESTRUTURA DEFENSIVA PADRÃO EM EXAME
public double calcularTotal(AidBox box) {
 if (box == null) return 0.0; // 1. Validar parâmetro

 Container[] containers = box.getContainers();
 if (containers == null) return 0.0; // 2. Validar array devolvido

 double total = 0.0;
 for (int i = 0; i < containers.length; i++) {
 if (containers[i] != null) { // 3. Validar elemento do array
 total += containers[i].getCapacity();
 }
 }
 return total;
}
```

## 2 Devolver Arrays com Tamanho Exato (Sem Nulos Sobrantes)

Quando um método devolve um array de objetos (ex: `Route[] generate(...)` ou `AidBox[] getAidBoxes()` ), o array **NUNCA** pode ter posições `null` no fim.

**Técnica em 2 Passos (Contar -> Criar -> Preencher):**

```
// Passo 1: Contar os elementos válidos
int count = 0;
for (int i = 0; i < totalBoxes; i++) {
 if (boxes[i] != null && boxes[i].needsCollection()) {
 count++;
 }
}

// Passo 2: Criar o array com o tamanho EXATO e preencher
AidBox[] result = new AidBox[count];
int idx = 0;
for (int i = 0; i < totalBoxes; i++) {
 if (boxes[i] != null && boxes[i].needsCollection()) {
 result[idx] = boxes[i];
 idx++;
 }
}
return result;
```

## 3 Redefinição Impecável do Método `equals()`

Se o enunciado pedir para redefinir o `equals()` numa classe (ex: `RefrigeratedVehicleImpl` ), decorre este modelo de memória:

```
@Override
public boolean equals(Object obj) {
 if (this == obj) return true; // 1. Mesma referência de memória
 if (obj == null || !(obj instanceof Vehicle)) return false; // 2. Nulo ou tipo incompatível
```

```
Vehicle other = (Vehicle) obj; // 3. Downcasting

if (this.code == null) {
 return other.getCode() == null;
}
return this.code.equals(other.getCode()); // 4. Comparação pelo identificador único
}
```

#### 4 Construtores em Herança com `super(...)`

- Na subclasse (ex: `RefrigeratedVehicleImpl extends VeiculoImpl`), a **primeira linha do construtor** tem de ser obrigatoriamente `super(code, maxCapacity);`.
- Inicialize os atributos específicos da subclasse logo a seguir.

#### 5 Atributos `enabled` e Estados por Defeito

Se o enunciado disser "O veículo possui um estado (`ENABLED`, `DISABLED`) inicializado como `ENABLED` por defeito":

- Declare um atributo booleano: `private boolean enabled;`
- No construtor, coloque: `this.enabled = true;`

#### 6 Lançamento e Captura de Exceções Customizadas

- Ao criar uma exceção customizada: `public class RouteException extends Exception { ... }`
- Ao sinalizar um erro de negócio num método: `throw new RouteException("Mensagem de erro");`
- Ao chamar um método que lança a exceção: usar o bloco `try-catch` e colocar o código de fecho no `finally`.

### 3. Checklist de Validação Final (Antes de Entregar a Prova)

Antes de dar o exame por concluído, reveja visualmente a sua folha de resposta com este checklist:

- ☐ **Zero `java.util`:** Verificou se não escreveu `ArrayList`, `List`, `Arrays.copyOf`, `Objects.equals` ou `Scanner` em lado nenhum?
- ☐ **Modificadores de Acesso:** Os atributos das classes estão todos como `private`?
- ☐ **Getters/Setters:** Todos os métodos das interfaces foram implementados com `@Override`?
- ☐ **Validação de Nulos:** Colocou `if (obj != null)` dentro dos ciclos `for`?
- ☐ **Instanciação de Arrays:** Usou `new Tipo[tamanho]` e não tentou adicionar diretamente sem instanciar o array?
- ☐ **Teoria com Exemplo:** Todas as perguntas teóricas incluem um excerto de código Java de 5-8 linhas?
- ☐ **Respostas Diretas:** A primeira frase de cada resposta teórica responde de forma inequívoca à pergunta?

Seguindo este guia, a aprovação com nota máxima está garantida! Boa sorte! 🚀